

# Standalone Research Roadmap for a Publishable Agent Memory Defense Paper

## Abstract

The strongest two-week paper is **not** another attack paper and **not** another vector-store-only RAG defense. It is a compact, defense-first systems paper that asks a sharper question: can a single, framework-agnostic runtime layer reduce poisoned writes, unsafe retrieval, contradiction-induced failures, and cross-agent propagation across several agent memory subsystems without destroying utility or scalability? That paper is independently publishable because the public literature is now rich enough to define the problem, but still fragmented enough that a clear cross-subsystem defense with real framework adapters and scalability evidence would fill an obvious gap. Public work already documents modular agent memory architectures, persistent memory stacks, and multiple attack families spanning indirect prompt injection, RAG poisoning, long-term memory injection, and multi-agent propagation. [1]

The best paper direction is a **memory firewall** that operates on the write path, the retrieval path, and the propagation path. The core claim should be modest but meaningful: a provenance-aware, quarantine-capable, risk-gated memory middleware can materially lower unsafe retrieval and propagation across public agent frameworks and four stable memory subsystems—vector/RAG memory, semantic fact memory, episodic/example memory, and shared coordination memory—while preserving clean-task success and exhibiting controlled scaling as memory size and agent count grow. That is a far better short-paper contribution than trying to invent a novel attack, a new benchmark from scratch, or a heavyweight formal system you cannot finish by May 16. [2]

*Scope note:* the attached unpublished draft was not retrievable in this chat session, so the roadmap below is intentionally grounded only in public literature, public frameworks, and public threat models. That constraint also makes the proposed paper cleaner and more standalone.

## Recommended paper direction

### Proposed paper

| Item                  | Recommendation                                                                                                                                                           |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Proposed title</b> | <b>MemSieve: A Provenance-Aware Memory Firewall for LLM Agents</b>                                                                                                       |
| <b>Exact problem</b>  | Defend mutable agent memory against contaminated writes, unsafe retrieval, contradiction collisions, and cross-agent propagation across heterogeneous memory subsystems. |
| <b>Central</b>        | A lightweight middleware layer over public memory APIs can reduce                                                                                                        |

| Item                               | Recommendation                                                                                                                                                                                                                                     |
|------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>hypothesis</b>                  | unsafe retrieval and propagation substantially, without large clean-task or latency penalties.                                                                                                                                                     |
| <b>Paper type</b>                  | Short systems/security paper with real implementations and stress-tested evaluation, not a theory paper and not a novel attack paper.                                                                                                              |
| <b>Memory subsystems in scope</b>  | Vector/RAG memory, semantic fact memory, episodic/example memory, shared coordination memory.                                                                                                                                                      |
| <b>Frameworks in MVP</b>           | LangGraph, LlamaIndex, CrewAI. AutoGen is a strong optional fourth. Letta/MemGPT is a stretch target.                                                                                                                                              |
| <b>Best venue fit in two weeks</b> | Realistically strongest as a compact systems/security paper or workshop/short-paper submission. It is <b>not</b> a credible 13-page main-track IEEE S&P / ACM CCS / USENIX Security package in two weeks unless you already have most of the code. |

The research problem should be stated exactly like this: **modern LLM agents increasingly treat memory as a writable runtime subsystem rather than a passive retrieval index, but current defenses are fragmented across corpus filtering, prompt separation, or tool-time checks; there is no compact, public, cross-framework demonstration that one middleware layer can make memory writes, memory retrieval, and memory propagation simultaneously safer across multiple memory types.**

That makes the paper independently publishable because the novelty is **not** “memory can be attacked.” Public work already established that. The publishable unit is instead: **a reusable defense architecture plus a public, cross-framework, scalability-first evaluation.** The paper stands on public sources alone and does not need any private benchmark, unpublished baseline, or unpublished scenario to make sense. [3]

This is also **not** merely a hidden rewrite of your private draft, by construction. The paper’s contribution is defined against public frameworks, documented public memory interfaces, public threat models, and new generic metrics. It should not reuse any unpublished attack narrative, private metric, or draft-specific setup. If you keep the title, threat model, scenarios, metrics, and baselines within the public space outlined below, reviewers can evaluate the paper without knowing anything about the private draft.

### Why this defense problem is timely

The defense problem is timely because the public agent ecosystem now exposes durable memory primitives as first-class features. LangGraph distinguishes short-term and long-term memory and explicitly discusses semantic, episodic, and procedural memory; LlamaIndex exposes a Memory class with FIFO short-term state and long-term memory blocks; CrewAI documents unified memory with semantic, recency, and importance scoring plus shared and scoped views; AutoGen exposes explicit memory query, update\_context, and add interfaces; and Letta/MemGPT uses core, archival, recall, and

shared memory abstractions. In other words, the architecture trend is moving toward **more writable state, more retrieval**, and **more cross-session persistence**, not less. [4]

At the same time, public security work already spans the relevant threat surface: indirect prompt injection through retrieved data, poisoning of RAG corpora, backdoor-style poisoning of memory or knowledge bases, query-only memory injection, self-reinforcing persistent compromise through long-term memory, and cross-agent infection dynamics. Benchmarks such as InjecAgent, AgentDojo, and Agent Security Bench show that these failures are not toy-only phenomena and that many current defenses remain brittle or narrowly scoped. That is exactly why a **defense-first memory middleware paper** is timely now. [5]

## Novelty and research gaps

### Core novelty

The paper’s novelty should be stated in one precise paragraph:

**MemSieve** is a framework-agnostic defense pipeline that treats agent memory as a **write–retrieve–propagate** system rather than only a retriever or a prompt. It normalizes heterogeneous memory records into a common schema with provenance, trust priors, contradiction signals, and lineage edges; validates or quarantines records at write time; reranks and diversifies candidates at retrieval time using risk-aware scoring; rewrites retrieved context into typed, provenance-marked evidence before reasoning; and blocks memory-conditioned tool actions or agent-to-agent propagation when support is weak, contradictory, or quarantine-tainted. The novelty is therefore the **combination of cross-subsystem defense coverage, public framework adapters, generic defense metrics, and central scalability evaluation**, not a new attack or a model-specific filter.

That novelty is real because current public work is split into disconnected islands. One island covers memory architectures and agent design; another covers attacks such as RAG poisoning, memory injection, and multi-agent infection; another covers inference-time defenses such as provenance cueing, structured queries, task-alignment gating, or traceback of poisoned RAG documents. What public work still does **not** provide is a compact, cross-framework demonstration that one lightweight layer can secure **multiple memory subsystems** and **propagation paths** under realistic scale pressures. [6]

### Concrete gaps in public research

| Gap                                      | Why existing public work is insufficient                                | How the proposed paper addresses it             | Evidence needed to convince reviewers       |
|------------------------------------------|-------------------------------------------------------------------------|-------------------------------------------------|---------------------------------------------|
| <b>Persistent memory is defended too</b> | PoisonedRAG and RAGForensics are valuable, but they are centered on RAG | Make persistent memory defense the main object: | Show strong results on at least one non-RAG |

| Gap                                                        | Why existing public work is insufficient                                                                                                                                                                                                                                                                                                   | How the proposed paper addresses it                                                                           | Evidence needed to convince reviewers                                   |
|------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| <b>narrowly</b>                                            | corpora; MINJA, AgentPoison, Zombie Agents, and other memory-poisoning work show persistent agent-memory compromise, yet public defenses remain architecture-specific or domain-specific. [7]                                                                                                                                              | semantic, episodic, and shared memory are first-class, not side cases.                                        | subsystem in the main table, not appendix.                              |
| <b>Cross-memory-subsystem security analysis is missing</b> | Public frameworks and CoALA-style taxonomies distinguish semantic, episodic, procedural, archival, and shared memory, but attack/defense papers usually evaluate one substrate at a time. [8]                                                                                                                                              | Cover four stable subsystems with one defense schema and one metric family.                                   | A single table that spans subsystems, not four disconnected mini-evals. |
| <b>Multi-agent propagation is underconnected to memory</b> | Prompt Infection and G-Safeguard show multi-agent infection and remediation, but the central object is conversation or utterance topology; memory-specific propagation across shared notes, summaries, and team memory remains understudied. [9]                                                                                           | Add lineage-aware propagation controls for shared memory and agent-to-agent write paths.                      | Show reduced propagation depth/fan-out as agent count grows.            |
| <b>Defense pipelines usually stop too early</b>            | Spotlighting, StruQ, f-secure systems, Task Shield, and RAGForensics attack different stages—source separation, structured input, system-level planning, task alignment, or traceback—but there is still no small, reusable defense that jointly covers write admission, retrieval gating, and action gating for memory-heavy agents. [10] | Make the design explicitly multi-stage: write-time, retrieval-time, pre-reasoning, pre-tool, and cross-agent. | Strong ablations showing that no single stage explains all gains.       |
| <b>Scalable, memory-centric</b>                            | InjecAgent, AgentDojo, and ASB are large public                                                                                                                                                                                                                                                                                            | Make scalability one of the                                                                                   | Separate scalability                                                    |

| Gap                       | Why existing public work is insufficient                                                                                                                                             | How the proposed paper addresses it | Evidence needed to convince reviewers                                       |
|---------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------|-----------------------------------------------------------------------------|
| <b>evaluation is weak</b> | benchmarks, but they are broad agent-security harnesses rather than dedicated memory-defense studies, and they do not make memory-scale or agent-count scaling the main result. [11] | paper's three headline claims.      | section with curves over memory size, agent count, poison ratio, and top-k. |

The harsh reviewer view is simple: if the paper only shows vector-store filtering, it will read as “another RAG defense.” If it only shows propagation in a tiny team, it will read as synthetic. The differentiator is **breadth across memory subsystems plus disciplined scalability evidence**.

## Defense design

### Threat model to freeze early

Use a crisp, public threat model and do **not** let it balloon.

The attacker should have four realistic capabilities, all already represented in public work: **query-only write inducement** that causes the agent to store malicious or misleading memory; **corpus or passage poisoning** of a retrievable memory source; **indirect prompt injection through retrieved or tool-returned content**; and **compromised peer-agent output** that gets written into shared or coordination memory. These capabilities are grounded in prior public work on indirect prompt injection, PoisonedRAG, AgentPoison, MINJA, Prompt Infection, and Zombie Agents. [12]

Keep the out-of-scope list short and explicit: model-weight poisoning, host compromise, embedding-model attacks, covert channels, prompt leakage from vendor internals, and human-in-the-loop social engineering. If you do not freeze scope, reviewers will say the threat model is vague.

### Memory subsystems to cover

Do **not** try to cover every conceivable memory category. The strongest paper covers **four** subsystems and treats procedural/scratchpad memory as an extension, not the MVP core.

| Subsystem                | Concrete representation in the paper      | Main attack pressure                    | Why it belongs in the MVP                      |
|--------------------------|-------------------------------------------|-----------------------------------------|------------------------------------------------|
| <b>Vector/RAG memory</b> | Retrieved passages, docs, notes, external | Poisoned passages, hidden instructions, | Public literature already shows this is a real |

| Subsystem                              | Concrete representation in the paper                                   | Main attack pressure                                                  | Why it belongs in the MVP                                                                                |
|----------------------------------------|------------------------------------------------------------------------|-----------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
|                                        | KB items                                                               | top-k domination                                                      | attack surface, and every reviewer understands it.                                                       |
| <b>Semantic memory</b>                 | User/project facts, profiles, stable preferences, long-term facts      | False fact insertion, policy overwrite, contradiction collisions      | This is where persistence and personalization create sticky failures.                                    |
| <b>Episodic memory</b>                 | Prior trajectories, demonstrations, successful examples, action traces | Unsafe plan replay, poisoned exemplars, imitation of bad sequences    | This is how poisoning affects agent behavior even without explicit tool prompts.                         |
| <b>Shared coordination memory</b>      | Crew notes, shared blocks, team branches, inter-agent summaries        | Cross-agent contamination, cascading summaries, shared-note infection | This is the most distinctive subsystem for propagation claims.                                           |
| <b>Procedural or scratchpad memory</b> | Editable prompt fragments, reflective rules, working summaries         | Rule drift and tool-policy edits                                      | Mention as an extension, but do not make it central unless a stable implementation falls out “for free.” |

This subsystem choice is defensible because public agent memory work and public frameworks already distinguish fact memory, experience memory, instruction memory, archival memory, and shared memory patterns. [13]

## Proposed system architecture

**Architecture in one sentence:** MemSieve sits between the framework’s memory API and the agent/runtime, forcing every memory event through a canonical record schema and a sequence of defensive gates.

| Stage                       | What it does                                                                                                                                                                                     | What it blocks or limits                                        |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| <b>Adapter layer</b>        | Wraps framework memory interfaces and converts writes/retrievals to a canonical record format: {content, subsystem, source_type, source_id, actor_id, timestamp, parent_ids, trust_prior, tags}. | Framework-specific fragmentation.                               |
| <b>Write-time admission</b> | Scores incoming records for provenance, instructionality, contradiction with high-trust memory, duplication/flooding, and sensitivity.                                                           | Poisoned insertions, false facts, hidden tool directives, near- |

| Stage                                | What it does                                                                                                                                                                                                        | What it blocks or limits                                            |
|--------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
|                                      | Either commits, down-trusts, or quarantines.                                                                                                                                                                        | duplicate floods.                                                   |
| <b>Quarantine and audit</b>          | Stores suspicious records in a separated namespace/store; keeps append-only audit logs of write/retrieve/decision events.                                                                                           | Silent contamination and poor forensics.                            |
| <b>Retrieval-time risk scoring</b>   | Computes utility score = relevance + trust + corroboration – contradiction – lineage risk – duplication; enforces source/lineage diversity in top-k.                                                                | Top-k domination, recency-biased poisoning, contradiction collapse. |
| <b>Pre-reasoning context shaping</b> | Renders retrieved memory as typed evidence blocks (trusted facts, trusted examples, shared notes, contradictions, quarantined excerpts) instead of raw free text.                                                   | Treating arbitrary retrieved text as executable instruction.        |
| <b>Pre-tool verification</b>         | Before tool execution, checks whether the action is explicitly user-authorized or corroborated by at least one trusted memory source; otherwise asks for confirmation, abstains, or degrades to read-only behavior. | Tool-use poisoning and memory-conditioned unsafe actions.           |
| <b>Propagation controller</b>        | Carries lineage across shared memory and agent-to-agent writes; decays trust on summarization hops; freezes rapid low-trust fan-out.                                                                                | Cross-agent infection and memory cascades.                          |

The key design choice is to avoid a monolithic learned detector. That is a schedule trap and a review trap. Use **interpretable, calibratable features** plus optional second-pass LLM checks only where necessary. In two weeks, the paper wins on architecture and evaluation discipline, not on a fancy classifier.

## What each stage should do

**At write time**, MemSieve should assign a source trust prior by origin: seeded system memory and curated corpus sources start high; user free text, web retrievals, tool outputs, and peer-agent summaries start lower; recursively summarized content receives decayed trust. It should also scan for instructionality, policy-edit language, near duplicates, and contradictions against high-trust semantic memory. High-risk records go to quarantine; medium-risk records stay committed but marked low-trust; low-risk records are committed normally.

**At retrieval time**, MemSieve should refuse to treat similarity as enough. It should retrieve candidates from the native framework, but rerank them by trust, corroboration, contradiction penalties, lineage penalties, and diversity constraints. This is where you make the paper feel different from “just another filter.” A reviewer should be able to see that retrieval now optimizes **task utility under risk**, not cosine similarity alone.



**Before reasoning**, retrieved memory should be transformed into structured evidence. The agent should see that some memory is “trusted factual memory,” some is “episodic example memory,” some is “shared note from another agent,” and some is “flagged contradictory or quarantined.” Low-trust content should enter as data to reason *about*, not as instructions to follow.

**Before tool execution**, the defense should verify that high-impact tool arguments are supported by either explicit user intent or corroborated trusted memory. If the action depends only on low-trust memory, the system should confirm or abstain. This keeps the contribution defense-first rather than retrieval-only.

**For multi-agent propagation**, every shared-memory write or inter-agent summary should carry parent record IDs and source trust. Trust should decay across summarization hops. Rapid fan-out of a low-trust claim should trigger a freeze or quarantine. That is how you convert “propagation” from hand-wavy rhetoric into a measurable systems claim.

## Why the design generalizes across frameworks

The generalization argument must be implementation-based, not philosophical. LangGraph documents short-term and namespace long-term memory with semantic/episodic/procedural framing; LlamaIndex documents explicit short-term and long-term memory objects; CrewAI exposes shared, scoped, and sliced memory with composite scoring; AutoGen exposes memory query, add, and update\_context; and Letta/MemGPT exposes core, archival, and shared memory paths. That means a thin middleware adapter is realistic across public stacks. [14]

## Baselines, scenarios, and metrics

### Public framework and defense baselines

Use **public, implementable** baselines. Do not burn a week on heroic reproduction.

#### *Framework comparison targets*

| Framework         | Why it is fair                                                                                                                                                                                                                                  | Two-week decision |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------|
| <b>LangGraph</b>  | Public docs explicitly define short-term and long-term memory and discuss semantic, episodic, and procedural memory, making it a strong “conceptually rich” baseline. [15]                                                                      | <b>Include</b>    |
| <b>LlamaIndex</b> | Public docs expose a concrete Memory class with short-term FIFO buffering and long-term memory blocks, which is ideal for adapter-based defense insertion. [16]                                                                                 | <b>Include</b>    |
| <b>CrewAI</b>     | Public docs expose unified memory, scoped views, shared crew memory, read-only slices, and composite scoring over semantic similarity, recency, and importance. That makes it the best public baseline for coordination-memory evaluation. [17] | <b>Include</b>    |



| Framework                    | Why it is fair                                                                                                                                                                                              | Two-week decision      |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------|
| <b>AutoGen</b>               | Public docs expose explicit memory interfaces and multi-agent runtime patterns, so it is a fair public target; however, the project's maintenance-mode status means integration risk is real. [18]          | <b>Optional fourth</b> |
| <b>Letta</b> / <b>MemGPT</b> | Public research and docs expose memory hierarchy, archival retrieval, and shared blocks, making it architecturally relevant for long-term memory; deployment/API complexity makes it a stretch target. [19] | <b>Stretch only</b>    |

**Harsh call:** the MVP should be **LangGraph + LlamaIndex + CrewAI**. Add AutoGen only if it works fast. Add Letta only if it is practically zero-friction. If Letta costs more than half a day, cut it.

#### *Defense comparison targets*

| Defense baseline                 | Why it is fair                                                                                                                                | Realistic in two weeks                      | Decision        |
|----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------|-----------------|
| <b>No defense</b>                | Necessary lower bound.                                                                                                                        | Trivial.                                    | <b>Include</b>  |
| <b>Provenance-only filter</b>    | Strong industry-style sanity baseline: tag source types, but no contradiction or action gating.                                               | Easy.                                       | <b>Include</b>  |
| <b>Spotlighting</b>              | Public provenance-cueing defense against indirect prompt injection with minimal utility loss. [20]                                            | Easy to adapt as retrieval-context wrapper. | <b>Include</b>  |
| <b>Trust-threshold sanitizer</b> | Matches the strongest lightweight current public memory-defense direction: trust-aware retrieval/sanitization and threshold calibration. [21] | Easy enough.                                | <b>Include</b>  |
| <b>RAGForensics</b>              | Public traceback defense for vector-store poisoning; fair for vector/RAG rows only. [22]                                                      | Moderate if code is usable.                 | <b>Optional</b> |
| <b>Task Shield</b>               | Public tool-time task-alignment defense; fair for action-gating comparison, not write-path defense. [23]                                      | Moderate.                                   | <b>Optional</b> |
| <b>StruQ</b>                     | Strong public defense idea, but it requires structured-query support plus a specially trained model. [24]                                     | Schedule trap.                              | <b>Exclude</b>  |

The mistake to avoid is obvious: a five-defense comparison with shaky reproductions is weaker than a three-defense comparison with clean ablations and real scaling results.

## Standalone attack and stress scenarios

Ground the scenario family in public attack categories—RAG poisoning, memory injection, indirect prompt injection, agent-poisoning of memory/knowledge bases, prompt infection, and persistent self-evolving memory compromise—but make the concrete scenarios generic, safe, and new. [25]

| Scenario                            | Attacker capability                                                          | Target subsystem  | Expected failure mode                                          | Success and defense-success criteria                                                                                                                           | Data to collect                                                               |
|-------------------------------------|------------------------------------------------------------------------------|-------------------|----------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| <b>Poisoned fact insertion</b>      | Query-only or low-trust content causes a false persistent fact to be stored. | Semantic memory   | Later answer or plan uses the false fact.                      | <b>Attack success:</b> poisoned fact is written and later retrieved/used. <b>Defense success:</b> fact is quarantined, down-trusted, contradicted, or ignored. | write outcome, trust score, retrieval presence, downstream answer correctness |
| <b>Retrieved instruction hijack</b> | Attacker inserts a poisoned chunk/document likely to be retrieved.           | Vector/RAG memory | Retrieved text is treated as instruction rather than evidence. | <b>Attack success:</b> unsafe retrieved context changes answer/tool plan. <b>Defense success:</b> chunk is quarantined, down-ranked, or rendered inert.        | top-k records, final context, risk scores, task outcome                       |
| <b>Experience graft</b>             | Attacker seeds an episodic/exam                                              | Episodic memory   | Agent imitates a bad prior plan or example.                    | <b>Attack success:</b> poisoned                                                                                                                                | retrieved exemplars, plan                                                     |

| Scenario                              | Attacker capability                                                                                   | Target subsystem           | Expected failure mode                        | Success and defense-success criteria                                                                                                                                | Data to collect                                                  |
|---------------------------------------|-------------------------------------------------------------------------------------------------------|----------------------------|----------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|
|                                       | ple store with a “successful” but unsafe trajectory.                                                  |                            |                                              | example is selected and copied.<br><b>Defense success:</b> exemplar is suppressed or contradicted.                                                                  | similarity, tool sequence, task result                           |
| <b>Conflict and recency collision</b> | Attacker injects a recent contradicting record that overlaps semantically with benign trusted memory. | Semantic or shared memory  | Recency/similarity wins over trust.          | <b>Attack success:</b> false branch selected without warning.<br><b>Defense success:</b> contradiction surfaced, trusted branch preferred, or abstention triggered. | contradiction flags, selected record, final answer/action        |
| <b>Shared-memory cascade</b>          | Compromised peer agent writes contaminated summary/note into shared memory.                           | Shared coordination memory | Contamination fans out to downstream agents. | <b>Attack success:</b> depth > 1 and downstream contaminate d write or action.<br><b>Defense success:</b> cascade is capped or frozen.                              | lineage graph, propagation depth, fan-out, downstream tool calls |
| <b>Memory flooding</b>                | Many near-duplicate low-                                                                              | Vector, episodic,          | Top-k domination,                            | <b>Attack success:</b>                                                                                                                                              | duplicate clusters,                                              |

| Scenario | Attacker capability         | Target subsystem | Expected failure mode                | Success and defense-success criteria                                                                                                                    | Data to collect                                  |
|----------|-----------------------------|------------------|--------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------|
|          | grade records are inserted. | or shared memory | degraded diversity, rising overhead. | attacker cluster dominates retrieval and hurts utility.<br><b>Defense success:</b> duplicate throttling and diverse retrieval keep safe utility stable. | source diversity in top-k, latency, task success |

Use safe stub tools and public or synthetic corpora. Do **not** bring in EHRs, autonomous driving, or external browsing if you can avoid it. Those domains create engineering drag and unnecessary reviewer scrutiny.

### Metric table

Let  $Q_a$  be attack queries,  $Q_b$  benign queries,  $P$  poisoned records,  $R(q)$  the retrieved set for query  $q$ , and  $C(q)$  the final memory-conditioned reasoning context.

| Metric                          | Formal definition                                                               | Why reviewers will care                             | How to collect it                                            | Which claim it supports |
|---------------------------------|---------------------------------------------------------------------------------|-----------------------------------------------------|--------------------------------------------------------------|-------------------------|
| <b>Attack Success Reduction</b> | $1 - \frac{ASR_{def}}{ASR_{base}}$                                              | Direct headline security number.                    | Evaluate attack-triggered failures with and without defense. | “The defense works.”    |
| <b>Unsafe Retrieval Rate</b>    | $URR = \frac{1}{ Q_a } \sum_{q \in Q_a} \mathbf{1}[R(q) \cap P \neq \emptyset]$ | Tells whether poisoned records still win retrieval. | Log retrieved IDs before reasoning.                          | “Retrieval is safer.”   |

| Metric                               | Formal definition                                                                                                              | Why reviewers will care                                             | How to collect it                                                    | Which claim it supports                                                  |
|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|----------------------------------------------------------------------|--------------------------------------------------------------------------|
| <b>Contaminated Context Rate</b>     | $CCR = \frac{1}{ Q } \sum_q \mathbf{1}[C(q) \text{ contains unquarantined po}]$                                                | Stronger than URR because it measures what the model actually sees. | Log post-defense context assembly.                                   | “Unsafe memory is not merely retrieved less; it reaches the model less.” |
| <b>Propagation Depth</b>             | Maximum or mean graph distance from initial poisoned write to downstream contaminated writes/actions in the memory lineage DAG | Makes “propagation” measurable.                                     | Build parent-child lineage edges across shared writes and summaries. | “Cross-agent spread is contained.”                                       |
| <b>Quarantine Precision / Recall</b> | Standard $TP/(TP + FP)$ and $TP/(TP + FN)$ over poisoned-vs-benign write labels                                                | Prevents “security by blocking everything.”                         | Use synthetic poison labels and conflict labels.                     | “The gate is selective, not dumb.”                                       |
| <b>Benign Utility Preservation</b>   | $BUP = \frac{U_{def}^{clean}}{U_{base}^{clean}}$                                                                               | Reviewers will punish large clean-task loss.                        | Measure clean task success or exact-match/goal completion.           | “Utility survives.”                                                      |
| <b>Task Success Under Defense</b>    | $TSD = U_{def}^{attack}$                                                                                                       | Reduction in attack success is                                      | Compute task completion on                                           | “Defense remains operational.”                                           |

| Metric                        | Formal definition                                                                                                                                             | Why reviewers will care                            | How to collect it                        | Which claim it supports                  |
|-------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------|------------------------------------------|------------------------------------------|
|                               |                                                                                                                                                               | meaningless if the agent also stops solving tasks. | attacked workloads.                      |                                          |
| <b>Latency Overhead</b>       | $LO = \frac{T_{def} - T_{base}}{T_{base}}$ using p50 and p95                                                                                                  | Central for deployability.                         | End-to-end request timing.               | “Overhead is acceptable.”                |
| <b>Memory Growth Overhead</b> | $MGO = \frac{S_{def} - S_{base}}{S_{base}}$                                                                                                                   | Provenance and audit metadata have storage cost.   | Measure bytes/records in store and logs. | “The defense scales in storage.”         |
| <b>Scalability Slope</b>      | Regression coefficient $\hat{\beta}$ in $Y = \alpha + \beta \log N$ for $Y \in \{\text{latency}, URR, CCR\}$ and $N \in \{\text{records}, \text{agents}, k\}$ | Better than cherry-picked endpoints.               | Fit slopes over sweep experiments.       | “Scaling is a first-class contribution.” |

If you need one extra diagnostic metric, add **Top-k Source Diversity**. If you need to cut, cut it first.

## Experimental and scalability plan

### Minimum publishable experiment set

The minimum publishable paper is **not** huge. It is:

- **Three frameworks:** LangGraph, LlamaIndex, CrewAI.
- **Four subsystems:** vector/RAG, semantic, episodic, shared coordination.
- **Four defense settings:** no defense, provenance-only, spotlighting-style retrieval wrapper, trust-threshold sanitizer, and full MemSieve.
- **Four scenario families:** poisoned fact insertion, retrieved instruction hijack, conflict/recency collision, shared-memory cascade, plus flooding as the main stress test.

- **One primary model:** use one low-cost but capable hosted model for the full matrix.
- **One optional transfer model:** only after the main figures are complete.
- **Three random seeds or retrieval randomizations** where applicable.
- **A clean utility suite and a scalability suite.**

That is enough for a short paper if the results are clean.

The strongest implementation order is:

1. **LangGraph + vector/semantic memory + write gate + retrieval gate**
2. **CrewAI shared memory + propagation controller**
3. **LlamaIndex episodic/long-term blocks**
4. **Metric logging and lineage graph**
5. **Only then** optional AutoGen

What can be simplified:

- Use one shared embedding backend across all frameworks.
- Use deterministic stub tools and auto-gradable tasks.
- Use a single canonical record schema everywhere.
- Use one public corpus family plus synthetic conflict injection rather than many domains.
- Keep the primary experimental emphasis on attack and defense, not absolute task intelligence.

What must **not** be skipped:

- A clean-task utility table
- An ablation table
- A propagation experiment
- A real scalability section
- At least one non-RAG subsystem result in the main paper

### Stronger optional experiment set

If time holds, the strongest upgrades are:

- Add **AutoGen** as a fourth framework.
- Add one **Letta/MemGPT-style** archival/shared-memory case if integration is painless.
- Add a **second model** for transfer robustness.
- Run a small **AgentDojo** or **ASB-inspired** subset only if it plugs in cleanly on day one; otherwise cite those benchmarks for positioning and move on. [\[26\]](#)



- Add optional comparisons to **RAGForensics** on vector rows and **Task Shield** on tool rows if their code is easy to reuse. [27]

Do **not** attempt:

- StruQ reproduction
- New model training
- Full AgentDojo integration from scratch
- A new benchmark from scratch
- Human evaluation
- A fifth framework

### Experiment matrix

| Experiment                   | Frameworks                    | Subsystems               | Baselines                                 | Primary metrics                 | Priority    |
|------------------------------|-------------------------------|--------------------------|-------------------------------------------|---------------------------------|-------------|
| <b>Clean utility</b>         | LangGraph, LlamaIndex, CrewAI | all four                 | all included defenses                     | BUP, TSD, LO                    | <b>Must</b> |
| <b>Write-path poisoning</b>  | LangGraph, LlamaIndex         | semantic, episodic       | no defense, threshold, full               | ASR reduction, quarantine P/R   | <b>Must</b> |
| <b>Unsafe retrieval</b>      | all MVP frameworks            | vector/RAG, semantic     | no defense, spotlighting, threshold, full | URR, CCR, TSD                   | <b>Must</b> |
| <b>Conflict resolution</b>   | LangGraph, CrewAI             | semantic, shared         | no defense, threshold, full               | contradiction win-rate, TSD     | <b>Must</b> |
| <b>Shared-memory cascade</b> | CrewAI, optional AutoGen      | shared coordination      | no defense, provenance-only, full         | propagation depth, fan-out, TSD | <b>Must</b> |
| <b>Memory flooding</b>       | LangGraph, LlamaIndex, CrewAI | vector, episodic, shared | no defense, threshold, full               | URR, LO, top-k diversity        | <b>Must</b> |
| <b>Memory-size scaling</b>   | all MVP frameworks            | all applicable           | threshold vs full                         | scalability slope, LO, URR      | <b>Must</b> |
| <b>Agent-count scaling</b>   | CrewAI, optional AutoGen      | shared coordination      | provenance-only vs full                   | propagation depth, fan-out, LO  | <b>Must</b> |
| <b>Defense ablations</b>     | best-performing 2             | all four                 | full minus one module                     | ASR, CCR, TSD                   | <b>Must</b> |

| Experiment                   | Frameworks            | Subsystems      | Baselines          | Primary metrics    | Priority        |
|------------------------------|-----------------------|-----------------|--------------------|--------------------|-----------------|
|                              | frameworks            |                 |                    |                    |                 |
| <b>Second-model transfer</b> | one or two frameworks | vector shared + | no defense vs full | ASR reduction, BUP | <b>Optional</b> |

## What results would actually convince reviewers

For a short paper, the following would be convincing:

- **At least 60–80% relative reduction** in unsafe retrieval or attack success on three or more scenario families.
- **At most 5–10% clean-task loss** on the main clean utility table.
- **A propagation result** where contamination depth or fan-out is sharply capped compared with the undefended case.
- **A p95 latency overhead that stays moderate** under realistic top-k values.
- **Ablations** showing that write-time gate, retrieval gate, and propagation controller each matter.
- **Scaling curves** showing that the full pipeline remains stable as memory size and agent count grow.

If the only strong result is vector-store filtering, the paper is weak. If the powerful result is shared-memory containment plus one other subsystem, the paper becomes much stronger.

## Scalability section design

Treat scalability as a headline contribution, not appendix filler.

### *Strong figure set*

| Figure title                                                    | X-axis                                                              | Y-axis                                             | Expected trend                                                                                                | Why it supports the defense claim                                          | Minimum data needed                                                                   |
|-----------------------------------------------------------------|---------------------------------------------------------------------|----------------------------------------------------|---------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| <b>Unsafe retrieval under memory growth and poison pressure</b> | total memory records on log scale; separate curves for poison ratio | unsafe retrieval rate or contaminated context rate | Undefended and weak baselines worsen or remain nontrivial; full MemSieve stays meaningfully lower and flatter | Shows the defense is not brittle to bigger stores or higher poison density | 4–5 memory sizes $\times$ 3–4 poison ratios $\times$ 3 defenses $\times$ 3 frameworks |

| Figure title                                                   | X-axis                                  | Y-axis                                                  | Expected trend                                                                                                   | Why it supports the defense claim                                     | Minimum data needed                                           |
|----------------------------------------------------------------|-----------------------------------------|---------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|---------------------------------------------------------------|
| <b>Cross-agent contamination as teams scale</b>                | number of agents                        | propagation depth or contaminated shared writes per run | Undefended growth approaches linear or worse; full defense caps spread or flattens slope                         | Makes the paper clearly about propagation, not only retrieval hygiene | 2, 4, 8, 16, 32 agents × 2 team workflows × 30 runs           |
| <b>Security-cost frontier across top-k and enabled modules</b> | p95 latency overhead or memory overhead | attack-success reduction or benign utility preservation | Some modules buy little security at high cost; full pipeline should dominate weak baselines in the useful region | Prevents the “too heuristic / too expensive” critique                 | top-k 2, 4, 8, 16 × key ablations × one main framework family |

These are much stronger than the usual weak “latency vs N” plot because they couple **security outcome** to **scale**, which is what reviewers actually care about in this topic.

## Execution and paper plan

### Two-week execution timeline

| Date         | Technical goal                                                                                                 | Writing / artifact goal                                | Hard rule                                                             |
|--------------|----------------------------------------------------------------------------------------------------------------|--------------------------------------------------------|-----------------------------------------------------------------------|
| <b>May 2</b> | Freeze title, problem statement, threat model, frameworks, subsystems, and baselines. Build literature matrix. | Draft abstract and intro claims.                       | No code before the claim is frozen.                                   |
| <b>May 3</b> | Smoke-test LangGraph, LlamaIndex, CrewAI. Define canonical record schema and logging format.                   | Draft threat model and related-work positioning notes. | If a framework does not run cleanly, do not debug for hours; move on. |
| <b>May 4</b> | Implement adapter layer and write-time gate for LangGraph.                                                     | Draft design subsection.                               | Get one end-to-end defended run today.                                |
| <b>May</b>   | Port adapter and write gate to                                                                                 | Draft system                                           | Keep the schema                                                       |

| Date          | Technical goal                                                                                                  | Writing / artifact goal            | Hard rule                                                                |
|---------------|-----------------------------------------------------------------------------------------------------------------|------------------------------------|--------------------------------------------------------------------------|
| <b>5</b>      | LlamaIndex and CrewAI.                                                                                          | architecture table.                | identical across frameworks.                                             |
| <b>May 6</b>  | Implement quarantine store, audit log, duplicate throttling, contradiction checks.                              | Draft metric definitions.          | No fancy detector training.                                              |
| <b>May 7</b>  | Implement retrieval-time reranking, source diversity, and safe context rendering.                               | Draft experimental setup.          | Produce first retrieval table today.                                     |
| <b>May 8</b>  | Implement pre-tool verification and lineage propagation control in CrewAI; basic shared-memory cascade harness. | Draft evaluation section shell.    | If propagation is not measurable, fix the logger before doing more runs. |
| <b>May 9</b>  | Run the <b>mock deadline</b> matrix: one model, three frameworks, two scenarios, three baselines.               | Freeze claims for version one.     | Do not expand scope after this point.                                    |
| <b>May 10</b> | Add episodic-memory scenario and flooding stress test. Optional AutoGen only if easy.                           | Start figures.                     | If AutoGen is messy, cut it immediately.                                 |
| <b>May 11</b> | Run clean utility and conflict/recency experiments.                                                             | Fill utility and ablation tables.  | Clean-task results are mandatory.                                        |
| <b>May 12</b> | Run memory-size sweeps and top-k sweeps.                                                                        | Draft scalability section.         | At least one strong scaling curve must exist by tonight.                 |
| <b>May 13</b> | Run agent-count propagation sweeps. Optional second model if main results are complete.                         | Refine figure captions and claims. | Do not add a second model before propagation is finished.                |
| <b>May 14</b> | Full metric calculation, error analysis, and ablation completion.                                               | Full paper draft complete.         | If a module has weak value, cut it and simplify.                         |
| <b>May 15</b> | Harsh internal review pass: shrink claims, fix threat model, tighten tables, remove fluff.                      | Near-final submission draft.       | Make the paper smaller and sharper, not broader.                         |
| <b>May 16</b> | Final submission cleanup: reproducibility notes, appendix, artifact packaging, naming consistency.              | Final submit.                      | No last-minute new experiments.                                          |

## Paper skeleton for a 6–8 page submission

| Section                            | Page budget | Exact purpose                                                                       | Claims to make                                                                                                            | Evidence figures /                                             | What to avoid                                    |
|------------------------------------|-------------|-------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------|--------------------------------------------------|
| <b>Title and Abstract</b>          | 0.2         | State the problem, defense, scope, and result in one breath.                        | Memory is writable state; MemSieve defends write, retrieve, and propagate.                                                | None beyond one sentence of quantitative summary if available. | Do not oversell “general immunity.”              |
| <b>Introduction</b>                | 0.8         | Explain why agent memory differs from ordinary RAG and why defenses are fragmented. | Public work shows memory is attackable; the missing piece is a unified defense middleware.                                | One motivating table or compact example; main result teaser.   | Do not spend the intro on attack taxonomy.       |
| <b>Background and Threat Model</b> | 0.8         | Define four subsystems and attacker capabilities.                                   | The threat is persistent contamination plus propagation, not just one bad chunk.                                          | Subsystem table; concise threat model paragraph.               | Do not make the threat model vague or universal. |
| <b>Defense Design</b>              | 1.4         | Present canonical schema and pipeline stages.                                       | Multi-stage gating beats single-stage fixes; provenance + contradiction + lineage are enough for a strong compact design. | Architecture table; optional scoring equation.                 | Do not dump implementation trivia.               |
| <b>Experimental Setup</b>          | 0.8         | Describe frameworks, tasks, baselines,                                              | The evaluation is public, reproducible, and fair.                                                                         | Framework/baseline table; metric table.                        | Do not overinflate benchmark breadth.            |

| Section                           | Page budget | Exact purpose                                                 | Claims to make                                                                        | Evidence figures /                   | What to avoid                                    |
|-----------------------------------|-------------|---------------------------------------------------------------|---------------------------------------------------------------------------------------|--------------------------------------|--------------------------------------------------|
|                                   |             | scenarios, metrics, models.                                   |                                                                                       |                                      |                                                  |
| <b>Evaluation</b>                 | 1.5         | Show attack reduction, clean utility, and ablations.          | The defense works and does not collapse utility.                                      | Main result table; ablation table.   | Do not bury clean-task losses.                   |
| <b>Scalability Analysis</b>       | 0.9         | Make scaling central.                                         | Security gains persist under larger stores and larger teams with acceptable overhead. | Three strong scalability figures.    | Do not show only raw latency.                    |
| <b>Discussion and Limitations</b> | 0.4         | Preempt reviewer attacks.                                     | The defense is not complete, but it is reusable and practical.                        | Error analysis or compact paragraph. | Do not pretend the system has formal guarantees. |
| <b>Related Work</b>               | 0.4         | Position against memory architectures, attacks, and defenses. | This is not another RAG defense and not merely a message-level defense.               | No figures needed.                   | Do not write a survey. Compress aggressively.    |
| <b>Conclusion</b>                 | 0.2         | Restate one clean claim.                                      | Memory security needs pipeline defense and scaling evidence.                          | None.                                | Do not introduce new ideas.                      |

## Contributions

- **We propose a framework-agnostic memory firewall for LLM agents** that secures the write, retrieval, and propagation paths of agent memory rather than only the retriever or the prompt.

- **We unify four practical agent memory subsystems**—vector/RAG, semantic fact memory, episodic/example memory, and shared coordination memory—under a single provenance-aware defense schema and evaluation methodology.
- **We introduce a scalable stress-evaluation harness for agent memory security** that measures unsafe retrieval, contaminated context, propagation depth, utility preservation, and overhead across public agent frameworks.
- **We show that memory defense must be evaluated as a scaling problem**, not just an attack-success problem, by varying memory size, poison ratio, agent count, subsystem count, and retrieval top-k.

## Reviewer-risk audit and final recommendation

Reviewer-risk table

| Reviewer risk                                       | Severity    | Why it is serious                                           | Fix                                                                                     | Evidence that reduces the risk                                      |
|-----------------------------------------------------|-------------|-------------------------------------------------------------|-----------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| <b>“This is just another RAG defense.”</b>          | <b>High</b> | This is the most likely kill shot.                          | Put non-RAG subsystems and shared-memory propagation in the main results, not appendix. | Main table must include semantic, episodic, and shared-memory wins. |
| <b>“The threat model is vague.”</b>                 | <b>High</b> | Security reviewers hate fuzzy attacker powers.              | Use the four-capability threat model above and a strict out-of-scope list.              | Threat model table with capability-to-scenario mapping.             |
| <b>“The defense is too heuristic.”</b>              | <b>High</b> | If the design looks like ad hoc filters, novelty collapses. | Use a principled, interpretable scoring pipeline and strong ablations.                  | Module ablation table and calibration / threshold sensitivity plot. |
| <b>“The experiments are too small.”</b>             | <b>High</b> | A tiny eval will not carry a systems-defense paper.         | Three frameworks minimum, four subsystems, multiple scale sweeps.                       | Experiment matrix and scaling curves.                               |
| <b>“The paper lacks real framework comparison.”</b> | <b>High</b> | Framework-agnostic claims require actual adapters.          | Use LangGraph, LlamaIndex, CrewAI for sure; AutoGen optional.                           | One framework comparison table with identical scenario templates.   |

| Reviewer risk                                              | Severity           | Why it is serious                                           | Fix                                                                                                                        | Evidence that reduces the risk                                           |
|------------------------------------------------------------|--------------------|-------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| <b>“The scalability results are synthetic.”</b>            | <b>Medium–High</b> | Purely synthetic scaling is easy to dismiss.                | Use real framework stores and actual multi-agent shared-memory flows.                                                      | Memory-size and agent-count figures with real runtime paths.             |
| <b>“The defense hurts benign task performance.”</b>        | <b>High</b>        | Reviewers will assume over-blocking unless shown otherwise. | Put clean utility preservation in the main paper.                                                                          | Clean-task success table and frontier plot.                              |
| <b>“The novelty is unclear.”</b>                           | <b>High</b>        | The topic is crowded.                                       | Make the novelty sentence brutally clear: write–retrieve–propagate defense across subsystems and frameworks, with scaling. | First paragraph of intro and first sentence of related work.             |
| <b>“Why not compare against stronger public defenses?”</b> | <b>Medium</b>      | Fair question, but schedule matters.                        | Include spotlighting and threshold sanitization; optionally add RAGForensics or Task Shield where fair.                    | Honest baseline table explaining include / optional / exclude decisions. |

## Final recommendation

The **single strongest paper direction** is:

### **MemSieve: A Provenance-Aware Memory Firewall for LLM Agents**

A defense-first, public, cross-framework paper centered on write-time validation, retrieval-time risk gating, pre-tool verification, and propagation containment across four memory subsystems.

This direction is better than the main alternatives:

| Alternative                      | Why it is weaker now                                                   |
|----------------------------------|------------------------------------------------------------------------|
| <b>Vector-store-only defense</b> | Too easy to dismiss as another RAG paper.                              |
| <b>Attack-only memory paper</b>  | Violates your defense-first goal and is crowded by public attack work. |
| <b>Propagation-only multi-</b>   | Risks looking synthetic unless tied back to persistent shared          |



| Alternative                 | Why it is weaker now                            |
|-----------------------------|-------------------------------------------------|
| <b>agent paper</b>          | memory and broader memory types.                |
| <b>Huge benchmark paper</b> | Impossible to finish convincingly in two weeks. |

The **minimum viable version** of the paper should be:

- LangGraph, LlamaIndex, CrewAI
- Vector/RAG, semantic, episodic, shared coordination memory
- No defense, provenance-only, spotlighting, trust-threshold sanitizer, full MemSieve
- Poisoned insertion, unsafe retrieval, contradiction collision, shared-memory cascade, flooding
- Clean utility table
- One ablation table
- Three scalability figures
- One primary model

The **stronger version if time allows** should add:

- AutoGen as a fourth framework
- Optional Letta/MemGPT archival/shared-memory case
- One second model
- Vector-row comparison with RAGForensics
- Tool-row comparison with Task Shield
- Small public benchmark subset if it plugs in cleanly

The harsh practical verdict is this: **build the smallest paper that undeniably shows cross-subsystem defense plus scaling**. That paper is much more likely to survive review than a broader but weaker project. The fastest way to kill the submission is to overreach on frameworks, domains, or baseline reproductions. The fastest way to make it publishable is to keep the claim narrow, the defense pipeline crisp, the metrics generic, and the scalability evidence central.

**Open questions / limitations:** because the unpublished draft was not retrievable here, I avoided overlap-specific judgments and built the roadmap entirely from public material. If you later notice any accidental similarity in naming, scenario framing, or metric phrasing, change those immediately before writing; your proposed paper is strong enough without leaning on anything private.

---

[1] [6] [8] [13] <https://arxiv.org/abs/2309.02427>

<https://arxiv.org/abs/2309.02427>

[2] [22] [27] <https://arxiv.org/abs/2504.21668>

<https://arxiv.org/abs/2504.21668>

[3] [7] [25] <https://arxiv.org/abs/2402.07867>

<https://arxiv.org/abs/2402.07867>

[4] [14] [15] <https://docs.langchain.com/oss/javascript/concepts/memory>

<https://docs.langchain.com/oss/javascript/concepts/memory>

[5] [12] <https://arxiv.org/abs/2302.12173>

<https://arxiv.org/abs/2302.12173>

[9] <https://arxiv.org/abs/2410.07283>

<https://arxiv.org/abs/2410.07283>

[10] [20] <https://arxiv.org/abs/2403.14720>

<https://arxiv.org/abs/2403.14720>

[11] <https://arxiv.org/abs/2403.02691>

<https://arxiv.org/abs/2403.02691>

[16] [https://docs.llamaindex.ai/en/stable/module\\_guides/deploying/agents/memory/](https://docs.llamaindex.ai/en/stable/module_guides/deploying/agents/memory/)

[https://docs.llamaindex.ai/en/stable/module\\_guides/deploying/agents/memory/](https://docs.llamaindex.ai/en/stable/module_guides/deploying/agents/memory/)

[17] <https://docs.crewai.com/en/concepts/memory>

<https://docs.crewai.com/en/concepts/memory>

[18] <https://microsoft.github.io/autogen/dev/user-guide/agentchat-user-guide/memory.html>

<https://microsoft.github.io/autogen/dev/user-guide/agentchat-user-guide/memory.html>

[19] <https://arxiv.org/abs/2310.08560>

<https://arxiv.org/abs/2310.08560>

[21] <https://arxiv.org/abs/2601.05504>

<https://arxiv.org/abs/2601.05504>

[23] <https://arxiv.org/abs/2412.16682>

<https://arxiv.org/abs/2412.16682>

[24] <https://arxiv.org/abs/2402.06363>

<https://arxiv.org/abs/2402.06363>

[26] <https://arxiv.org/abs/2406.13352>

<https://arxiv.org/abs/2406.13352>